# 实验 6　指针实验

## 6.1、实验目的

（1）熟练掌握指针的说明、赋值、使用。

（2）掌握用指针引用数组的元素，熟悉指向数组的指针的使用。

（3）熟练掌握字符数组与字符串的使用，掌握指针数组及字符指针数组的用法。

（4）掌握指针函数与函数指针的用法。

（5）掌握带有参数的 main 函数的用法。

## 6.2、实验题目及要求

### 1、源程序改错题

在下面所给的源程序中，函数 strcopy(t, s)的功能是将字符串 s 复制给字符串 t，并且返回串 t 的首地址。请单步跟踪程序，根据程序运行时出现的现象或观察到的字符串的值，分析并排除源程序的逻辑错误，使之能按照要求输出如下结果：

Input a string:

programming↙　（键盘输入）

programming

Input a string again:

language↙　（键盘输入）

language

```
#include<stdio.h>
char *strcopy(char *, const char *);
int main(void)
{
    char *s1, *s2, *s3;
    printf("Input a string:\n", s2);
    scanf("%s", s2);
    strcopy(s1, s2);
    printf("%s\n", s1);
```

```
    printf("Input a string again:\n", s2);
    scanf("%s", s2);
    s3 = strcopy(s1, s2);
    printf("%s\n", s3);
    return 0;
}
/*将字符串 s 复制给字符串 t，并且返回串 t 的首地址*/
char * strcopy(char *t, const char *s)
{
    while(*t++ = *s++);
    return (t);
}
```

**解答：**

（1） 错误修改：

1.s1,s2 没有初始化

2. strcop 不能返回 t 的首地址，因为 t 的地址已经被移动了

（2）修改后的代码：

```
#include <stdio.h>
char *strcopy(char *, const char *);
int main(void)
{
    char s1[100], s2[100];
    char *s3;
    printf("Input a string:\n");
    scanf("%s", s2);
    strcopy(s1, s2);
    printf("%s\n", s1);
    printf("Input a string again:\n");
    scanf("%s", s2);
    s3 = strcopy(s1, s2);
    printf("%s\n", s3);
    return 0;
}
```

```
/*将字符串 s 复制给字符串 t，并且返回串 t 的首地址*/
char *strcopy(char *t, const char *s)
{
    char *start = t;
    while (*t++ = *s++)
        ;
    return (start);
}
```

**2、源程序完善、修改替换题**

（1）下面程序中函数 strsort 用于对字符串进行升序排序，在主函数中输入 N 个字符串（字符串长度不超过 49）存入通过 malloc 动态分配的存储空间，然后调用 strsort 对这 N 个串按字典序升序排序（提示：使用 strcmp 函数比较字符（串）大小）。

①请在源程序中的下划线处填写合适的代码来完善该程序。

```
#include<stdio.h>
#include<_________>
#include<string.h>
#define N 4
/*对指针数组 s 指向的 size 个字符串进行升序排序*/
void strsort(char *s[], int size)
{
    _______temp;
    int i, j;
    for(i=0; i<size-1; i++)
        for (j=0; j<size-i-1; j++)
            if (____________)
            {
                temp = s[j];
                  ____________;
                s[j+1] = temp;
            }
}

int main()
```

```
{
    int i;
    char *s[N], t[50];
    for (i=0; i<N; i++)
    {
        gets(t);
        s[i] = (char *)malloc(strlen(t)+1);
        strcpy(_______);
    }
    strsort(_________);
    for (i=0; i<N; i++)     {puts(s[i]); free(s[i]);}
    return 0;
}
```

**解答：**

源代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define N 4
/*对指针数组 s 指向的 size 个字符串进行升序排序*/
void strsort(char *s[], int size)
{
    char *temp;
    int i, j;
    for (i = 0; i < size - 1; i++)
        for (j = 0; j < size - i - 1; j++)
            if (strcmp(s[j], s[j + 1]) > 0)
            {
                temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
}

int main()
```

```
{
    int i;
    char *s[N], t[50];
    for (i = 0; i < N; i++)
    {
        gets(t);
        s[i] = (char *)malloc(strlen(t) + 1);
        strcpy(s[i], t);
    }
    strsort(s, N);
    for (i = 0; i < N; i++)
    {
        puts(s[i]);
        free(s[i]);
    }
    return 0;
}
```

②数组作为函数参数其本质类型是指针。例如，对于形参 char *s[]，编译器将其解释为 char **s，两种写法完全等价。请用二级指针形参重写 strsort 函数，并且在该函数体的任何位置都不允许使用下标引用。

**解答：**

源代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define N 4
/*对指针数组 s 指向的 size 个字符串进行升序排序*/
void strsort(char *s[], int size)
{
    char *temp;
    int i, j;
    char **p = s;
    char **q;
```

```
    for (i = 0; i < size - 1; i++)
    {
        p = s;
        for (j = 0; j < size - i - 1; j++)
        {
            q = p + 1;
            if (strcmp(*p, *q) > 0)
            {
                temp = *p;
                *p = *q;
                *q = temp;
            }
            p++;
        }
    }
}

int main()
{
    int i;
    char *s[N], t[50];
    for (i = 0; i < N; i++)
    {
        gets(t);
        s[i] = (char *)malloc(strlen(t) + 1);
        strcpy(s[i], t);
    }
    strsort(s, N);
    for (i = 0; i < N; i++)
    {
        puts(s[i]);
        free(s[i]);
    }
    return 0;
}
```

（2）下面源程序通过函数指针和菜单选择来调用库函数实现字符串操作；串复制 strcpy、串连接 strcat 或串分解 strtok。

①请在源程序中的下划线处填写合适的代码来完善该程序，使之能按照要求输出下面结果：

1 copy string.

2 connect string.

3 parse string.

4 exit.

input a number (1-4) please!

2↙ （键盘输入）

input the first string please!

the more you learn,↙ （键盘输入）

input the second string please!

the more you get. ↙ （键盘输入）

the result is the more you learn, the more you get.

```
# include<stdio.h>
# include<string.h>
int main (void)
{
    ______________________;
    char a[80], b[80], *result;
    int choice;
    while(1)
    {
        do
        {
            printf("\t\t1 copy string.\n");
            printf("\t\t2 connect string.\n");
            printf("\t\t3 parse string.\n");
            printf("\t\t4 exit.\n");
```

```
            printf("\t\tinput a number (1-4) please.\n");
            scanf("%d", &choice);
        }while(choice<1 || choice>4);
        switch(choice)
        {
            case 1:   p = strcpy;    break;
            case 2:   p = strcat;    break;
            case 3:   p = strtok;    break;
            case 4:   goto down;
        }
        getchar();
        printf("input the first string please!\n");
        ________________________;
        printf("input the second string please!\n");
        ________________________;
        result = ____________(a, b);
        printf("the result is %s\n", result);
    }
down:
return 0;
}
```

**解答：**

源代码：

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char *(*p)(char *, const char *);
    char a[80], b[80], *result;
    int choice;
    while (1)
    {
        do
        {
            printf("\t\t1 copy string.\n");
```

```
            printf("\t\t2 connect string.\n");
            printf("\t\t3 parse string.\n");
            printf("\t\t4 exit.\n");
            printf("\t\tinput a number (1-4) please.\n");
            scanf("%d", &choice);
        } while (choice < 1 || choice > 4);
        switch (choice)
        {
        case 1:
            p = strcpy;
            break;
        case 2:
            p = strcat;
            break;
        case 3:
            p = strtok;
            break;
        case 4:
            goto down;
        }
        getchar();
        printf("input the first string please!\n");
        scanf("%s", a);
        printf("input the second string please!\n");
        scanf("%s", b);
        result = p(a, b);
        printf("the result is %s\n", result);
    }
down:
    return 0;
}
```

②函数指针的一个用途是用户散转程序，即通过一个转移表（函数指针数组）来实现多分枝函数处理，从而省去了大量的 if 语句或者 switch 语句。转移表中存放了各个函数的入口地址（函数名），根据条件的设定来查表选择执行相应的

函数。请使用转移表而不是 switch 语句重写以上程序。

**解答：**

源代码：

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char *(*p[])(char *, const char *) = {strcpy, strcat, strtok};
    char a[80], b[80], *result;
    int choice;
    while (1)
    {
        do
        {
            printf("\t\t1 copy string.\n");
            printf("\t\t2 connect string.\n");
            printf("\t\t3 parse string.\n");
            printf("\t\t4 exit.\n");
            printf("\t\tinput a number (1-4) please.\n");
            scanf("%d", &choice);
        } while (choice < 1 || choice > 4);
        if (choice == 4)
            goto down;
        char *(*func)(char *, const char *) = p[choice - 1];
        getchar();
        printf("input the first string please!\n");
        scanf("%s", a);
        printf("input the second string please!\n");
        scanf("%s", b);
        result = func(a, b);
        printf("the result is %s\n", result);
    }
down:
    return 0;
}
```

**3、编程设计题**

以下（1）至（4）题对应 Educoder 教学平台“C 语言实验”课程，实验 6，第 13 关实验 6-1、第 14 关实验 6-2、第 15 关实验 6-3， 以及第 16 关实验 6-4。

（1）一个长整型变量占 4 个字节，其中每个字节又分成高 4 位和低 4 位。输入一个长整型变量，要求从高字节开始，依次取出每个字节的高 4 位和低 4 位并以十六进制数字字符的形式进行显示，通过指针取出每字节。

样例输入：15

样例输出：0000000F

1）解答思路：算法流程如图 6-1 所示：

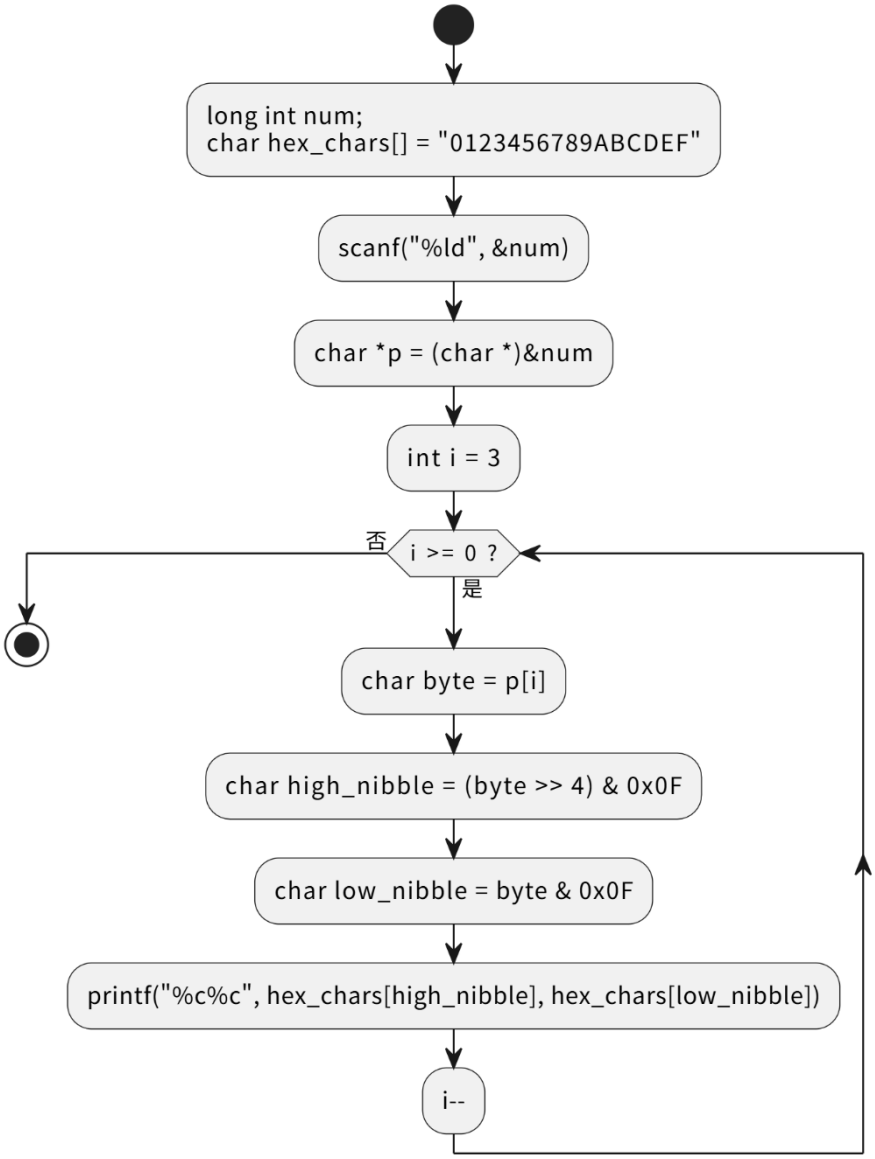


图 6-1 编程题 1 的程序流程图

2）源代码清单：

```
#include <stdio.h>

int main()
{
    long int num;
    char hex_chars[] = "0123456789ABCDEF";
    scanf("%ld", &num);
    char *p = (char *)&num;
    for (int i = 3; i >= 0; i--)
    {
        char byte = p[i];
        char high_nibble = (byte >> 4) & 0x0F;
        char low_nibble = byte & 0x0F;
        printf("%c%c", hex_chars[high_nibble], hex_chars[low_nibble]);
    }

    return 0;
}
```

3）测试

（a）测试数据：

如表 6-1 所示。

表 6-1 编程题 1 的测试数据

| 测试用例 | 程序输入 | 理论结果 | 实际结果 |
|---|---|---|---|
| 用例 1 | 15 | 0000000F | 0000000F |
| 用例 2 | 127 | 0000007F | 0000007F |

（b）对应测试测试用例的运行结果如图 6-2 所示。

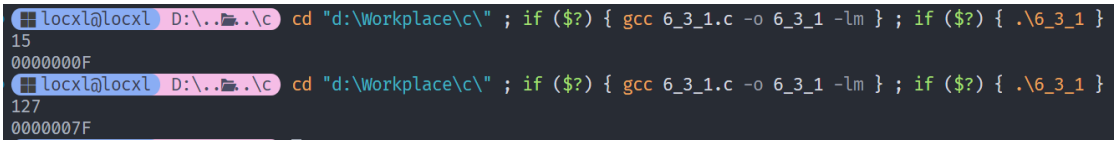


图 6-2 编程题 1 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（2）旋转是图像处理的基本操作，编程实现一个将一个图像逆时针旋转 90°。提示：计算机中的图像可以用一个矩阵来表示，旋转一个图像就是旋转对应的矩阵。将旋转矩阵的功能定义成函数，通过使用指向数组元素的指针作为参数使该

函数能处理任意大小的矩阵。要求在 main 函数中输入图像矩阵的行数 n 和列数 m，接下来的 n 行每行输入 m 个整数，表示输入的图像。输出原始矩阵逆时针旋转 90° 后的矩阵。

样例输入：

2　3

1　5　3

3　2　4

样例输出：

3　4

5　2

1　3

解答：

1） 解题思路：算法流程如图 6-3 所示。

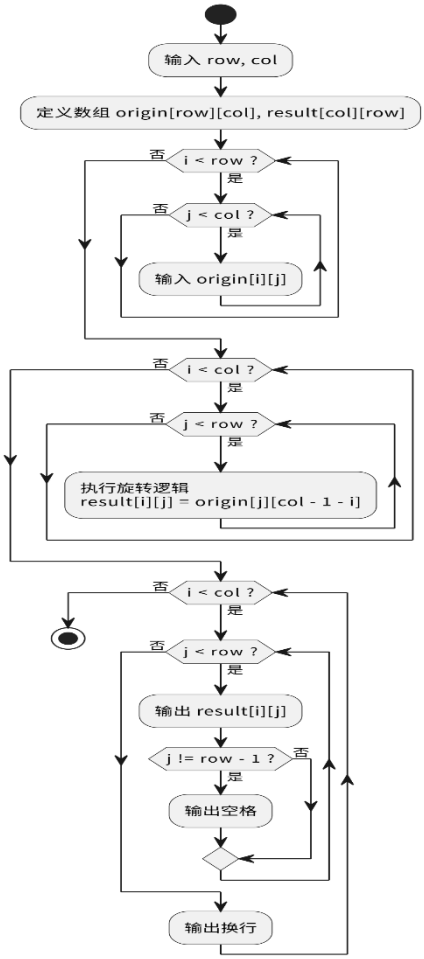


图 6-3 编程题 2 的程序流程图

1） 源代码清单：

```
#include <stdio.h>

int rotateMatrix(int row, int col, int origin[row][col], int result[col][row]);

int main()
{
	int row, col;

	scanf("%d %d", &row, &col);
	int origin[row][col];
	int result[col][row];
	for (int i = 0; i < row; i++)
	{
		for (int j = 0; j < col; j++)
		{
			scanf("%d", &origin[i][j]);
		}
	}
	rotateMatrix(row, col, origin, result);
	return 0;
}

int rotateMatrix(int row, int col, int origin[row][col], int result[col][row])
{
	for (int i = 0; i < col; i++)
	{
		for (int j = 0; j < row; j++)
		{
			result[i][j] = origin[j][col - 1 - i];
		}
	}
	for (int i = 0; i < col; i++)
	{
		for (int j = 0; j < row; j++)
```

```
            {
                printf("%d", result[i][j]);
                if (j != row - 1)
                {
                    printf(" ");
                }
            }
            printf("\n");
        }
        return 0;
}
```

2） 测试

（a） 测试数据：

如表 6-2 所示

**表 6-2 编程题 2 的测试数据**

| 测试用例 | 程序输入 | 理论结果 | 实际结果 |
|---|---|---|---|
| 用例 1 | 2 3<br>1 2 3<br>5 3 6 | 3 6<br>2 3<br>1 5 | 3 6<br>2 3<br>1 5 |
| 用例 2 | 3 2<br>1 2<br>4 6<br>7 9 | 2 6 9<br>1 4 7 | 2 6 9<br>1 4 7 |

（b） 对应测试测试用例的运行结果如图 6-4 所示。

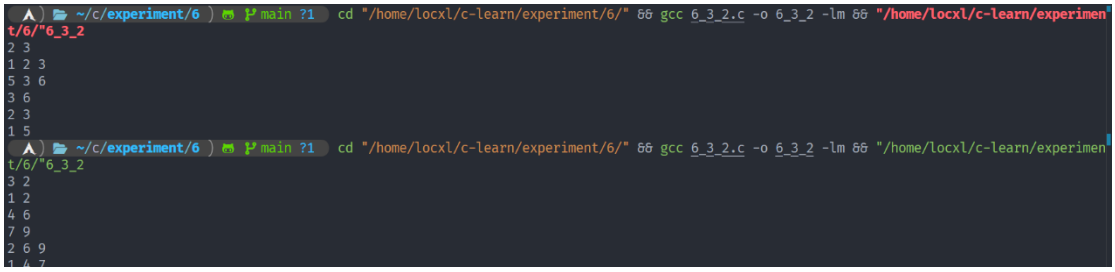


**图 6-4 编程题 2 测试用例的运行结果**

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（3）输入 n 行文本，每行不超过 80 个字符，用字符指针数组指向键盘输入的

n 行文本，且 n 行文本的存储无冗余，删除每一行中的前置空格（' '）和水平制表符（'\t'）。要求：将删除一行文本中前置空格和水平制表符的功能定义成函数，在 main 函数中输出删除前置空格符的各行。

解答：

1）解题思路：算法流程如图 6-5 所示。

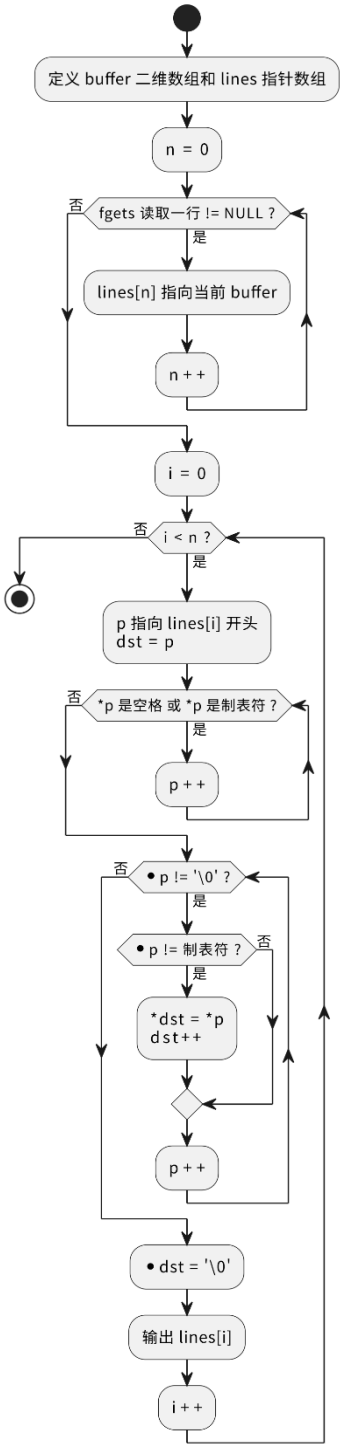


图 6-5 编程题 3 的程序流程图

2）源代码清单：

```
#include <stdio.h>

void removeSpace(char *line, int n)
{
    char *p = line;
    char *dst = p;

    while (*p == ' ' || *p == '\t')
    {
        p++;
    }
    while (*p)
    {
        if (*p != '\t')
        {
            *dst++ = *p;
        }
        p++;
    }
    *dst = '\0';
}

int main()
{
    char *lines[100];
    int max = 80;
    char buffer[100][80];
    int n = 0;
    while (fgets(buffer[n], max, stdin) != NULL)
    {
        lines[n] = buffer[n];
        n++;
    }
```

```
        for (int i = 0; i < n; i++)
        {
            removeSpace(lines[i], max);
            printf("%s", lines[i]);
        }
        return 0;
}
```

3） 测试

（a）测试数据：

如表 6-3 所示

**表 6-3 编程题 3 的测试数据**

| 测试用例 | 程序输入 | 理论结果 | 实际结果 |
|---|---|---|---|
| 用例 1 | faaf jlklj<br>jlfasdk klfas | faafjlklj<br>jlfasdk klfas | faafjlklj<br>jlfasdk klfas |
| 用例 2 | fsajflajl jlkfdas<br>jhkfldsaj<br>jflask | fsajflajl jlkfdas<br>jhkfldsaj jflask | fsajflajl jlkfdas<br>jhkfldsaj jflask |

（b）对应测试测试用例的运行结果如图 6-6 所示。

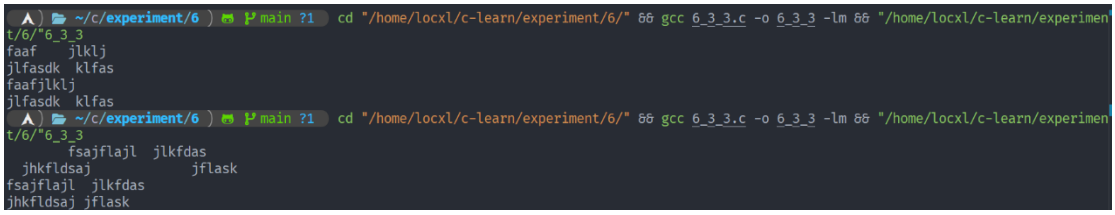


**图 6-6 编程题 3 测试用例的运行结果**

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（4）编写 8 个任务函数，一个 scheduler 调度函数和一个 execute 执行函数。仅在 main 函数中调用 scheduler 函数，scheduler 函数要求用最快的方式调度执行用户指定的任务函数。

①先设计 task0, task1, task2, task3, task4, task5, task6, task7 共 8 个任务函数，每个任务函数的任务就是输出该任务被调用的字符串。例如，第 0 个任务函数输出“task0 is called!”，第 1 个任务函数输出“task1 is called!”，以此类推。

②scheduler 函数根据键盘输入的数字字符的先后顺序，一次调度选择对应的

任务函数。例如，输入：1350 并回车，则 scheduler 函数一次调度选择 task1, task3, taks5, task0，然后以函数指针数组和任务个数为参数将调度选择结果传递给 execute 函数并调用 execute 函数。

③execute 函数根据 scheduler 函数传递的指针数组和任务个数为参数，按照指定的先后顺序依此调用执行选定的任务函数。

例如，当输入 13607122 并回车，程序运行结果如下：

task1 is called!

task3 is called!

task6 is called!

task0 is called!

task7 is called!

task1 is called!

task2 is called!

task2 is called!

**解答：**

1）解答思路：算法流程如图 6-7 所示：

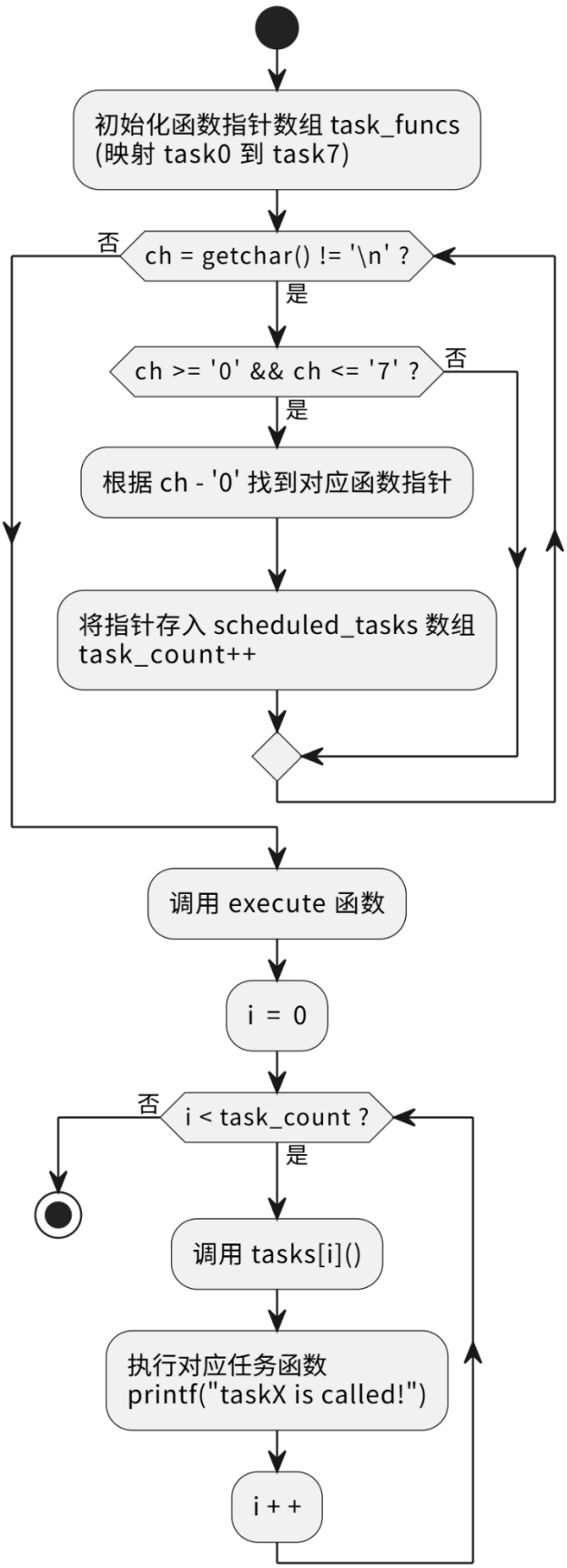


图 6-7 编程题 4 的程序流程图

2）源代码清单：

```
#include <stdio.h>
void task0()
{
	printf("task0 is called!\n");
}
void task1()
{
	printf("task1 is called!\n");
}
void task2()
{
	printf("task2 is called!\n");
}
void task3()
{
	printf("task3 is called!\n");
}
void task4()
{
	printf("task4 is called!\n");
}
void task5()
{
	printf("task5 is called!\n");
}
void task6()
{
	printf("task6 is called!\n");
}
void task7()
{
	printf("task7 is called!\n");
}
void execute(void (*tasks[])(), int task_count)
```

```
{
    for (int i = 0; i < task_count; i++)
    {
        tasks[i]();
    }
}
void scheduler()
{
    void (*task_funcs[8])() = {task0, task1, task2, task3, task4, task5, task6, task7};
    void (*scheduled_tasks[100])();
    int task_count = 0;
    char ch;
    while ((ch = getchar()) != '\n' && ch != EOF)
    {
        if (ch >= '0' && ch <= '7')
        {
            scheduled_tasks[task_count++] = task_funcs[ch - '0'];
        }
    }
    execute(scheduled_tasks, task_count);
}
int main()
{
    scheduler();
    return 0;
}
```

3）测试

（a）测试数据：

如表 4-1 所示。

**表 4-1 编程题 1 的测试数据**

| 测试用例 | 程序输入 | 理论结果 | 实际结果 |
|---|---|---|---|
| 用例 1 | 13607122 | task1 is called!<br>task3 is called!<br>task6 is called! | task1 is called!<br>task3 is called!<br>task6 is called! |

| | | | |
|---|---|---|---|
| | | task0 is called!<br>task7 is called!<br>task1 is called!<br>task2 is called!<br>task2 is called! | task0 is called!<br>task7 is called!<br>task1 is called!<br>task2 is called!<br>task2 is called! |
| 用例 2 | 4756235 | task4 is called!<br>task7 is called!<br>task5 is called!<br>task6 is called!<br>task2 is called!<br>task3 is called!<br>task5 is called! | task4 is called!<br>task7 is called!<br>task5 is called!<br>task6 is called!<br>task2 is called!<br>task3 is called!<br>task5 is called! |

（b）对应测试测试用例的运行结果如图 6-8 所示。

图 6-8 编程题 4 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

## 6.3 实验小结

本次实验主要加深了对指针、指针数组及函数指针的理解与应用。在实验过程中，我首先遇到了指针未初始化直接使用导致的程序崩溃问题（如源程序改错题中的 s1 和 s2），通过将其修改为字符数组或使用 malloc 动态分配内存，我深刻理解了指针变量必须指向有效内存空间才能赋值的原则；其次，在编写字符串复制和排序函数时，曾因在循环中直接移动指针导致丢失首地址或逻辑混乱，我采用了定义辅助指针保存起始地址的方法解决了该问题，并学会了用二级指针替代下标来处理字符串排序；在处理函数指针数组实现任务调度时，初次接触复杂的类型定义让我感到困惑，通过查阅资料并对比 switch 结构，我成功实现了转移表功能。

通过本次实验，我深刻体会到指针是 C 语言的灵魂，它能高效地处理内存（如矩阵旋转）和简化代码结构（如替代多分支语句），但也极易引发内存泄漏或越界错误，因此在使用指针时必须严谨对待地址运算与内存管理。